STM – DOCKER E KUBERNETES

Instrutoria Interna

Arquitetura Kubernetes: Control Plane, Workers e Pods

A Solução para Orquestração, Alta Disponibilidade e Gerenciamento Declarativo

Wescley de Macêdo Sousa
wescleysousa@stm.jus.br

INTRODUÇÃO AO KUBERNETES E ORQUESTRAÇÃO DE WORKLOADS

SUMÁRIO

MÓDULO 1: DA CONTAINERIZAÇÃO À ORQUESTRAÇÃO **3**
1.1 LIMITAÇÕES DO GERENCIAMENTO MANUAL DE CONTAINERS 3
1.2 O CONCEITO DE ORQUESTRAÇÃO DE CONTAINERS 3
1.3 O CONCEITO DE CLUSTER 4
1.4 A MATRIZ DE DIFERENCIAÇÃO: DOCKER VS. KUBERNETES 6
MÓDULO 2: ARQUITETURA DETALHADA DO KUBERNETES **8**
2.1 OS COMPONENTES DO CONTROL PLANE 8
2.2 OS COMPONENTES DOS WORKER NODES 10
2.3 FERRAMENTA DE CLIENTE: kubectl 11
MÓDULO 3: ABSTRAÇÕES E PRIMITIVOS FUNDAMENTAIS DO KUBERNETES **12**
3.1 OBJETOS KUBERNETES E A FILOSOFIA DECLARATIVA 12
3.2 ESTRUTURA E SINTAXE DE MANIFESTOS YAML 12
3.3 NAMESPACES (ISOLAMENTO LÓGICO NO CLUSTER) 13
3.4 PODS: A MENOR UNIDADE IMPLANTÁVEL 14
MÓDULO 4: CRIAÇÃO DO AMBIENTE PRÁTICO COM KIND (KUBERNETES IN DOCKER) **15**
4.1 O QUE É O KIND? 15
4.2 INSTALAÇÃO DO KIND E DEPENDÊNCIAS 16
4.3 PROVISIONANDO UM CLUSTER MULTI-NODE COM KIND 17
MÓDULO 5: LABORATÓRIO PRÁTICO GUIADO (PASSO A PASSO) **18**
5.1 ETAPA 1: INSPEÇÃO E MAPEAMENTO DO CLUSTER 18
5.2 ETAPA 2: CRIAÇÃO DECLARATIVA DO PRIMEIRO POD 21
5.3 ETAPA 3: INSPEÇÃO PROFUNDA E DIAGNÓSTICO (TROUBLESHOOTING) 22
5.4 ETAPA 4: EXCLUSÃO E LIMPEZA DE RECURSOS 25
MÓDULO 6: GUIA DE CONSULTA RÁPIDA (CHEATSHEET) **27**
REFERÊNCIAS **29**

MÓDULO 1: DA CONTAINERIZAÇÃO À ORQUESTRAÇÃO

1.1 LIMITAÇÕES DO GERENCIAMENTO MANUAL DE CONTAINERS

A adoção do Docker e da containerização resolveu o clássico problema de "funciona na minha máquina, mas não em produção" ao empacotar aplicações e suas dependências em unidades isoladas e reproduzíveis. No entanto, quando uma arquitetura evolui de dezenas para centenas ou milhares de containers distribuídos em ambientes de produção, o gerenciamento manual ou baseado em scripts simples torna-se inviável.

Os Gargalos do Gerenciamento Manual:

1. **Escalabilidade Manual e Rigidez de Hospedagem:** No Docker isolado, se um container atinge o limite de CPU ou memória, o operador precisa intervir manualmente para criar novas instâncias em outros servidores hospedeiros, configurar balanceadores de carga externos e mapear portas.
2. **Ausência de Autocura (*Self-Healing* Native):** Se o servidor físico ou a máquina virtual que hospeda o Docker Daemon falhar, todos os containers naquela máquina ficarão inacessíveis. O Docker Daemon pode reiniciar um container cujo processo principal morreu (`restart: always`), mas não consegue migrar esse container para *outro hospedeiro saudável* caso a máquina física apresente uma falha de hardware ou perda de conectividade de rede.
3. **Complexidade em Deployments Zerando Downtime (*Rolling Updates* e *Rollbacks*):** Atualizar uma aplicação para uma nova versão sem interromper o tráfego de usuários exige a criação manual do novo container, o redirecionamento progressivo de rede, a validação de saúde e a destruição do container antigo. Caso a nova versão apresente um bug crítico, o processo de reversão (*rollback*) manual é lento e propenso a falhas humanas.
4. **Gerenciamento de Estado e Descoberta de Serviços (*Service Discovery*):** Em ambientes dinâmicos onde containers sobem e descem constantemente, seus endereços IP mudam. Gerenciar registros DNS estáticos, arquivos de configuração ou atualizações manuais em proxies reversos para apontar para os IPs corretos inviabiliza a operação em escala.

1.2 O CONCEITO DE ORQUESTRAÇÃO DE CONTAINERS

A **Orquestração de Containers** é o processo de automatização do ciclo de vida completo de aplicações containerizadas em ambientes distribuídos.

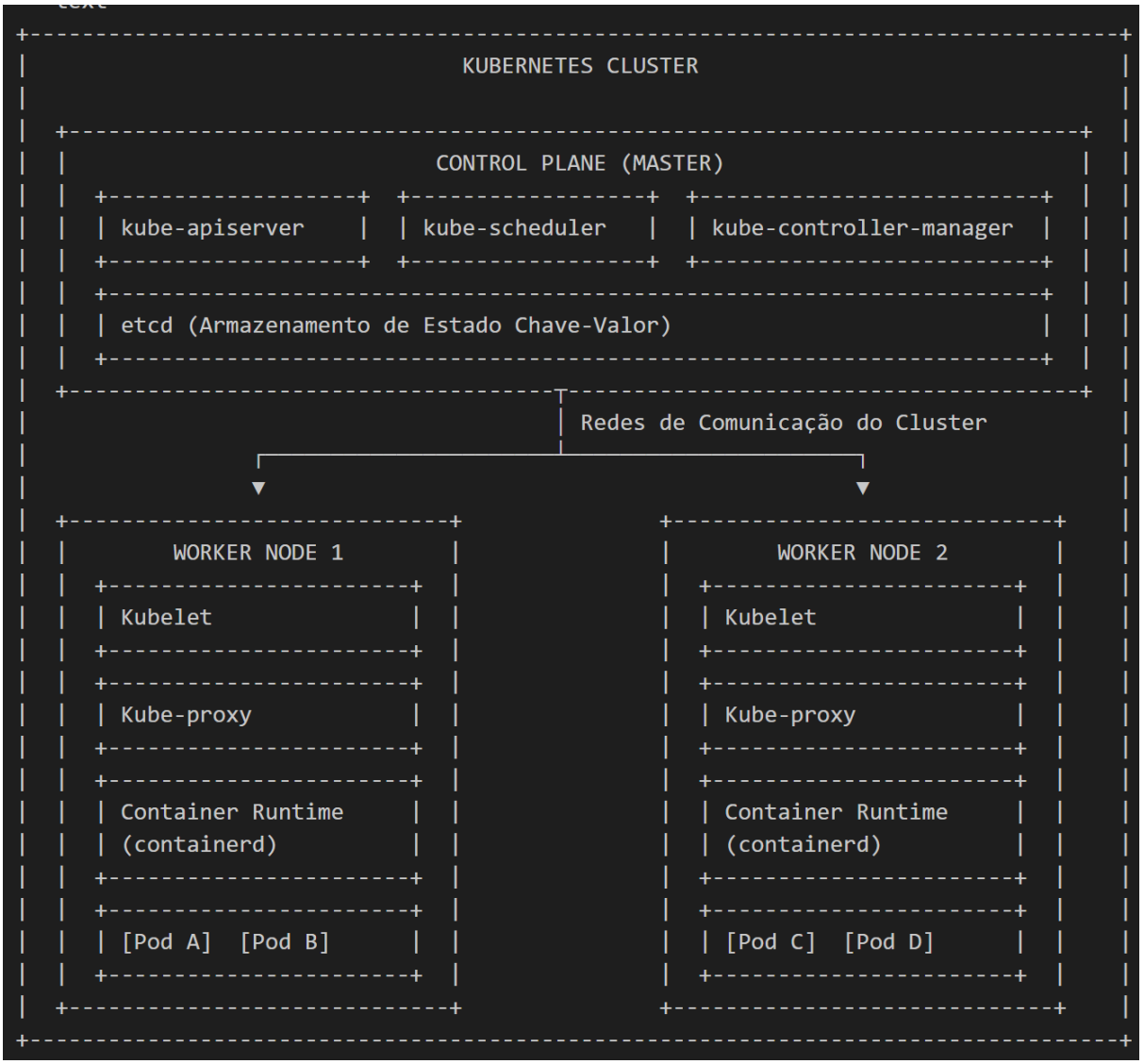
O orquestrador atua como um sistema operacional distribuído: ele abstrai a infraestrutura física ou virtual subjacente (servidores compute, storage e redes) e a transforma em um único *pool* unificado de recursos.

Responsabilidades Principais do Orquestrador:

- **Provisionamento e Agendamento (*Scheduling*):** Seleção automática da melhor máquina para alocar cada container com base em requisitos de recursos (CPU, Memória, GPU), afinidade e restrições de infraestrutura.
- **Autocura (*Self-Healing*):** Monitoramento contínuo da saúde dos containers e dos nós. Caso um container falhe, ele é reiniciado; caso um nó caia, os containers são remanejados automaticamente para nós saudáveis.
- **Escalonamento Automático (*Autoscaling*):** Aumento ou diminuição do número de instâncias de containers com base em métricas de uso de recursos (CPU/Memória) ou métricas de negócio (requisições por segundo).
- **Descoberta de Serviços e Balanceamento de Carga:** Atribuição de endereços virtuais e nomes DNS estáveis para conjuntos de containers, distribuindo o tráfego de rede de forma homogênea.
- **Gerenciamento Declarativo de Estado:** O operador não executa comandos do tipo "crie o container X agora". Em vez disso, o operador declara o **Estado Desejado** (*Desired State*) do sistema em um manifesto, e o orquestrador trabalha continuamente em uma malha de controle (*Reconciliation Loop*) para alinhar o **Estado Atual** (*Current State*) ao estado desejado.

1.3 O CONCEITO DE CLUSTER

Um **Cluster** no contexto do Kubernetes é um conjunto de máquinas computacionais — físicas ou virtuais (VMs) — interconectadas em rede, que trabalham em conjunto sob a coordenação do Kubernetes para executar aplicações containerizadas.

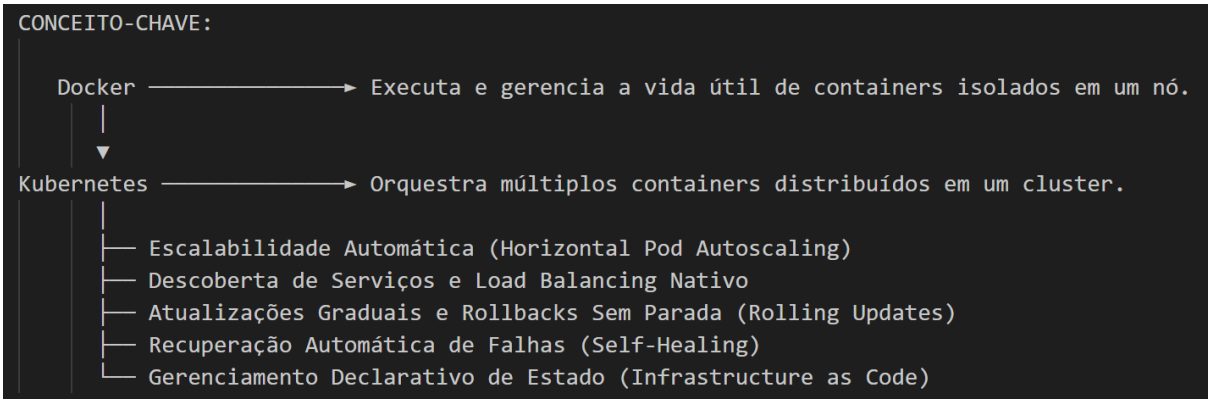


O cluster é dividido funcionalmente em duas camadas distintas:

1. **Control Plane (Plano de Controle):** Atua como o cérebro do cluster. Responsável por tomar decisões globais (como agendamento de tarefas), detectar eventos no cluster e responder a eles.
2. **Worker Nodes (Nós de Trabalho):** Máquinas encarregadas de efetivamente executar os containers da aplicação sob o comando e supervisão do Control Plane.

1.4 A MATRIZ DE DIFERENCIAÇÃO: DOCKER VS. KUBERNETES

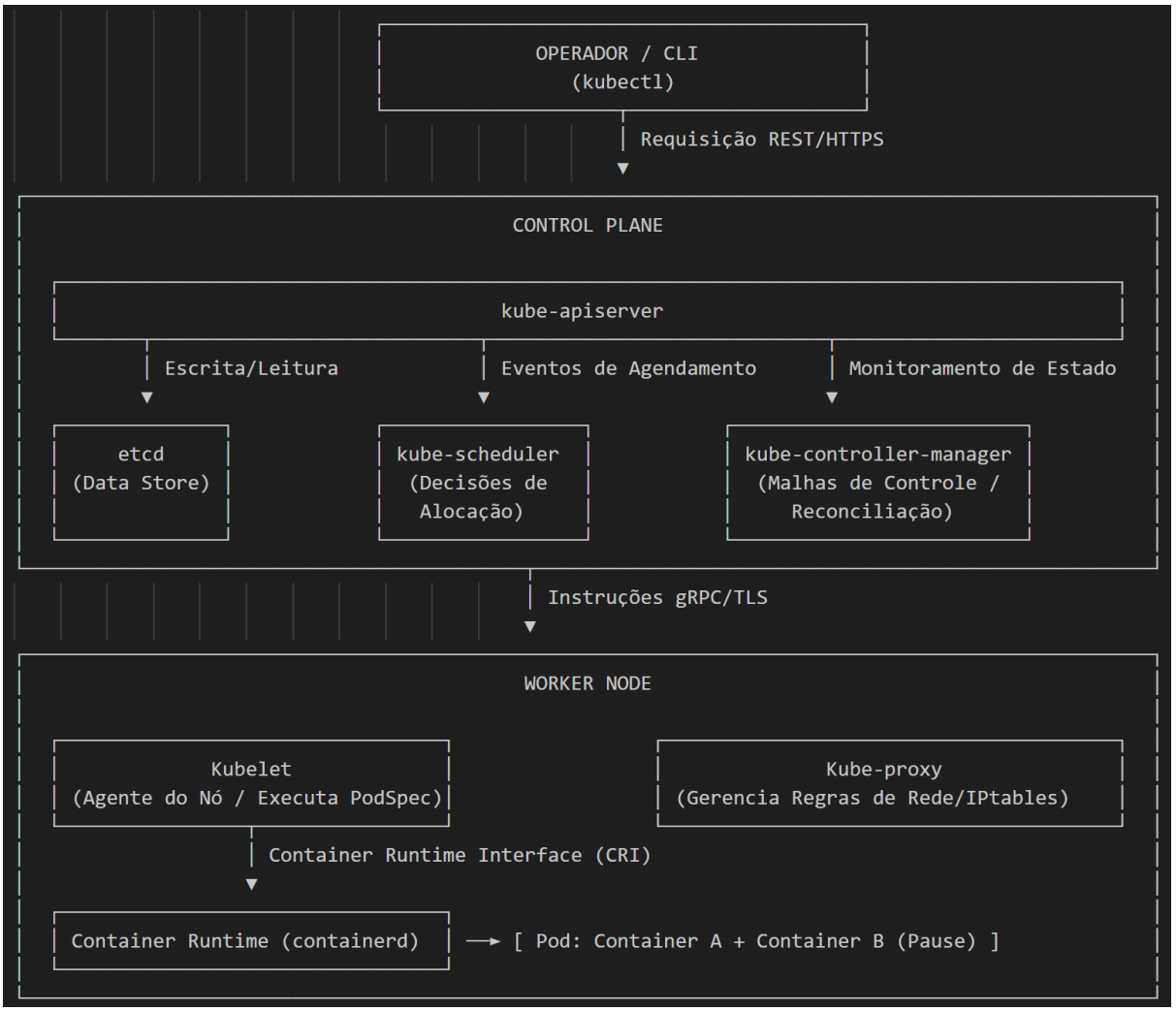
É comum que iniciantes confundam os papéis de Docker e Kubernetes. Eles não são tecnologias concorrentes diretas, mas sim **complementares**.



Funcionalidade / Característica	Docker (Docker Engine isolado)	Kubernetes (K8s)
Escopo Principal	Criar, construir e executar containers em um **único hospedeiro**.	Orquestrar, gerenciar e escalar aplicações em um **cluster de hospedeiros**.
Unidade Básica de Implantação	Container Docker (`docker run`).	Pod (Abstração que pode conter um ou mais containers).
Gerenciamento de Estado	Imperativo (Comandos manuais no terminal) ou declarativo local (`compose`).	**Declarativo nativo** com reconciliação contínua de estado.
Autocura (*Self-Healing*)	Reinicia processos caídos no *mesmo* nó. Não trata falha física da máquina.	Re-agenda e recria pods em **nós saudáveis** em caso de falha de hardware.
Escalonamento	Manual (Redimensionamento manual de containers via CLI).	**Automático e dinâmico** (*Horizontal Pod Autoscaler* e *Cluster Autoscaler*).
Descoberta de Serviços	Mapeamento de portas locais e DNS interno básico do Compose.	**DNS interno robusto (CoreDNS)**, IP Virtual estável (*Services*) e *Ingress*.
Deployments e Rollbacks	Processo manual propenso a interrupções de tráfego.	Nativos, automatizados, controlados por estratégia *RollingUpdate*.

MÓDULO 2: ARQUITETURA DETALHADA DO KUBERNETES

Para compreender o funcionamento interno do Kubernetes, é indispensável analisar detalhadamente a divisão entre o **Control Plane** e os **Worker Nodes**, examinando o papel exato de cada um dos seus componentes fundamentais.



2.1 OS COMPONENTES DO CONTROL PLANE

O Control Plane gerencia o estado global do cluster, intercepta requisições, toma decisões de alocação e responde a eventos de infraestrutura.

1. `kube-apiserver` (O Ponto de Entrada Central)

O **API Server** é o componente central do Control Plane. Ele expõe a API HTTP/JSON do Kubernetes e serve como a **única porta de entrada** para todas as interações com o cluster — seja por usuários humanos através do `kubectl`, por serviços internos ou pelos Worker Nodes.

- **Responsabilidades:**
  - **Autenticação e Autorização:** Valida credenciais (Tokens, Certificados X.509, OIDC) e verifica permissões via **RBAC** (*Role-Based Access Control*).
  - **Validação e Admissão:** Inspeciona a estrutura dos manifestos YAML submetidos e aplica políticas de admissão (*Admission Controllers*).
  - **Comunicação com o Banco de Dados:** É o **único** componente do cluster autorizado a se comunicar diretamente com o `etcd`. Nenhum Worker Node ou outro componente do Control Plane acessa o `etcd` diretamente.

2. `etcd` (O Armazenamento de Estado do Cluster)

O **etcd** é um banco de dados chave-valor, altamente disponível, consistente e distribuído, desenvolvido sob o algoritmo de consenso **Raft**.

- **Responsabilidades:**
  - Funciona como a **fonte única da verdade** (*Single Source of Truth*) para todo o cluster Kubernetes.
  - Armazena todas as especificações de objetos (Pods, Services, Roles, ConfigMaps, Secrets) e seus status atuais.
  - Todo e qualquer dado persistido no cluster é gravado no `etcd`. **Backup do cluster Kubernetes significa fazer backup do etcd.**

3. `kube-scheduler` (O Agente de Alocação)

O **Scheduler** monitora constantemente o `kube-apiserver` à procura de Pods recém-criados que ainda não foram atribuídos a nenhum nó hospedeiro (estado *Unassigned*).

- **Responsabilidades:**
  - Avalia os requisitos do Pod (quantidade de CPU/Memória requisitada, volumes necessários, *tolerations*, afinidade de nó).
  - Filtra os nós do cluster que atendem aos requisitos (*Filtering/Predicates*).
  - Classifica e pontua os nós elegíveis (*Scoring/Priorities*) para selecionar o nó ideal para executar aquele Pod específico.

- Notifica o `kube-apiserver` sobre a decisão de alocação (*Binding*).

4. `kube-controller-manager` (O Gerenciador de Malhas de Controle)

Executa os processos de controle (*Controllers*) em um único executável. Um controller é um loop infinito que lê o estado desejado gravado no `etcd` via API Server, compara com o estado real reportado pelos nós e executa ações corretivas para alcançar a convergência de estado (*Reconciliation Loop*).

- **Exemplos de Controllers Internos:**
  - **Node Controller:** Monitora a saúde dos nós e responde quando um nó para de enviar batimentos cardíacos (*heartbeats*).
  - **Replication Controller:** Garante que o número correto de cópias de um Pod esteja rodando no cluster em qualquer instante.
  - **EndpointSlice Controller:** Popula os objetos de endpoints conectando Services aos Pods correspondentes.
  - **ServiceAccount Controller:** Cria contas de serviço padrão para novos Namespaces.

2.2 OS COMPONENTES DOS WORKER NODES

Os Worker Nodes fornecem o ambiente de execução para as aplicações dos usuários.

1. `kubelet` (O Agente Principal do Nó)

O **Kubelet** é um agente de software instalado e executado como um serviço do sistema operacional em **cada nó** do cluster.

- **Responsabilidades:**
  - Registra o nó no cluster Kubernetes ao inicializar.
  - Recebe as especificações do Pod (*PodSpec*) enviadas pelo `kube-apiserver`.
  - Garante que os containers descritos na especificação sejam iniciados e continuem rodando de forma saudável.
  - Interage diretamente com o **Container Runtime** local através da especificação **CRI** (*Container Runtime Interface*).
  - Executa verificações de saúde (*Liveness, Readiness e Startup Probes*) nos containers e reporta o status atual do nó e dos Pods de volta para o `kube-apiserver`.

2. `Container Runtime` (O Executor de Containers)

O **Container Runtime** é o software subjacente responsável por baixar imagens, criar namespaces de isolamento do Linux, montar cgroups e efetivamente iniciar e parar os processos dos containers.

- O Kubernetes suporta qualquer runtime que cumpra a especificação **CRI** (*Container Runtime Interface*).
- O runtime mais utilizado no ecossistema atual do Kubernetes é o **`containerd`** (originalmente extraído do Docker), além do **`CRI-O`**.

3. `kube-proxy` (O Proxy de Rede do Nó)

O **Kube-proxy** é o componente responsável pela gestão da rede de serviços em cada nó do cluster.

- **Responsabilidades:**
  - Mantém as regras de rede (usando `iptables` ou `IPVS` no Linux) em cada nó hospedeiro.
  - Permite que as abstrações de rede do Kubernetes (os objetos do tipo *Service*) possuam endereços IP Virtuais estáveis (ClusterIP) e façam o balanceamento de carga de tráfego L4 (TCP/UDP) para os Pods corretos no backend.

2.3 FERRAMENTA DE CLIENTE: `kubectl`

O **`kubectl`** é a ferramenta de linha de comando (*CLI*) oficial para interagir com o Kubernetes. Ele lê um arquivo de configuração estruturado (geralmente localizado em `~/.kube/config`, conhecido como **Kubeconfig**).

O `kubectl` converte os comandos informados pelo operador em requisições REST/HTTPS formatadas em JSON/YAML e as envia para o `kube-apiserver` do cluster ativo.

```bash
# Exemplo do fluxo interno do kubectl
kubectl get pods

# O que ocorre nos bastidores:
# GET https://<apiserver-ip>:6443/api/v1/namespaces/default/pods
# Authorization: Bearer <token-do-kubeconfig>
```

MÓDULO 3: ABSTRAÇÕES E PRIMITIVOS FUNDAMENTAIS DO KUBERNETES

3.1 OBJETOS KUBERNETES E A FILOSOFIA DECLARATIVA

No Kubernetes, um **Objeto** é uma entidade persistente no sistema que representa o estado desejado de uma parte do cluster.

O Modelo Declarativo: Spec vs. Status

Todo manifesto de objeto Kubernetes é estruturado em dois campos principais de primeiro nível:

- **`spec` (Especificação):** Definido pelo operador humanamente. Declara o **Estado Desejado** (*Desired State*) — por exemplo, "quero 3 réplicas deste Pod com a imagem Nginx 1.25".
- **`status`:** Gerenciado exclusivamente pelo Kubernetes. Informa o **Estado Atual** (*Current State*) obtido pelo cluster em tempo real (ex: "atualmente existem apenas 2 réplicas rodando, 1 está sendo baixada").

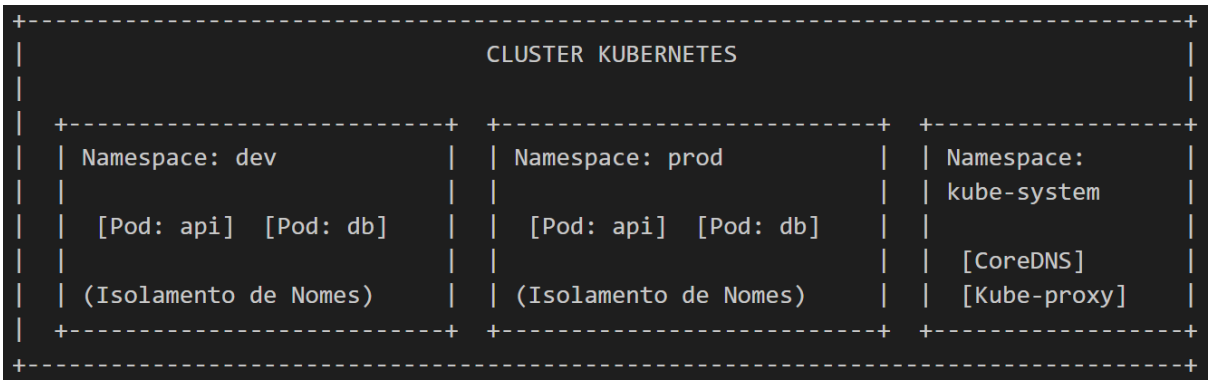
3.2 ESTRUTURA E SINTAXE DE MANIFESTOS YAML

Todos os arquivos de configuração declarativos do Kubernetes são escritos no formato YAML e exigem quatro campos obrigatórios no topo da estrutura:

```yaml
apiVersion: v1              # 1. Versão da API do Kubernetes que contém o objeto
kind: Pod                   # 2. Tipo de Objeto sendo criado
metadata:                   # 3. Dados de identificação (nome, rótulos, namespace)
  name: meu-pod-exemplo
  labels:
    app: web
    ambiente: producao
spec:                       # 4. O Estado Desejado (Especificação técnica do objeto)
  containers:
    - name: nginx-container
      image: nginx:1.25-alpine
      ports:
        - containerPort: 80
```

3.3 NAMESPACES (ISOLAMENTO LÓGICO NO CLUSTER)

Um **Namespace** é uma abstração para dividir um único cluster físico em múltiplos **clusters virtuais lógicos**. Ele fornece um escopo de nomes para os objetos, permitindo a separação de recursos entre equipes, projetos ou ambientes.



Namespaces Nativos Padrão:

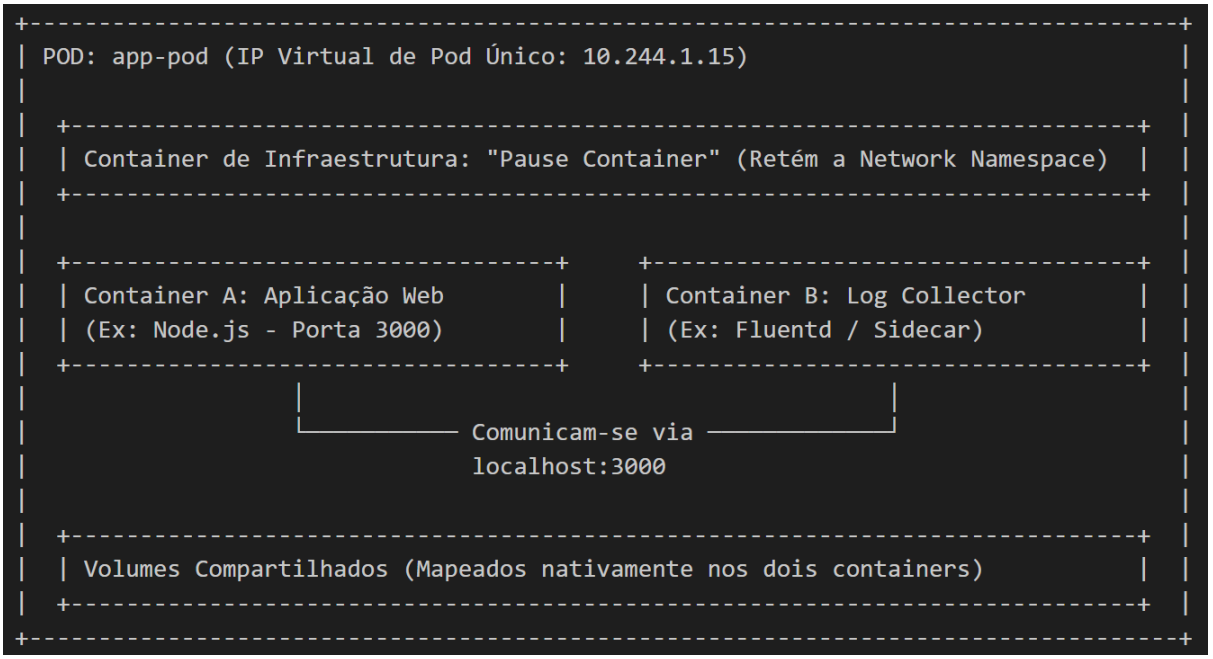
- **`default`:** O namespace padrão utilizado quando nenhum outro é informado no manifesto ou contexto.

- **`kube-system`:** Onde ficam alocados os componentes do próprio Kubernetes (como CoreDNS, Kube-proxy, CNI de rede).
- **`kube-public`:** Namespace de leitura pública, criado automaticamente para conter dados acessíveis por todo o cluster (como informações do cluster).
- **`kube-node-lease`:** Contém objetos do tipo *Lease* associados aos nós, usados pelo Kubernetes para determinar a saúde e presença do nó de forma performática.

3.4 PODS: A MENOR UNIDADE IMPLANTÁVEL

No Kubernetes, **você nunca roda um container diretamente**. A menor e mais fundamental unidade de abstração computacional é o **Pod**.

Um Pod é um invólucro (*wrapper*) que encapsula um ou mais containers que compartilham estritamente os mesmos recursos de **Rede**, **Armazenamento** e **IPC** (Comunicação Interprocessos).



Características Fundamentais do Pod:

1. **Endereçamento IP Único por Pod:** Todo Pod recebe um IP Virtual único dentro do cluster. Todos os containers dentro do mesmo Pod compartilham esse endereço IP e a mesma pilha de rede (*Network Namespace*).

2. **Comunicação em `localhost`:** Como os containers de um mesmo Pod compartilham a interface de rede, o *Container A* pode fazer requisições para o *Container B* apontando diretamente para `http://localhost:<porta>`.
3. **Containers de Apoio (*Sidecar Pattern*):** Embora a maioria dos Pods siga o padrão "1 Pod : 1 Container", o modelo multi-container permite acoplar processos utilitários (como coletores de logs, proxies de segurança ou sincronizadores de arquivos) que necessitam rodar estritamente colocalizados com a aplicação principal.
4. **O "Pause Container" (Infra Container):** Internamente, o Kubernetes instancia um container invisível chamado `pause` para cada Pod. Esse container reserva os *namespaces* do Linux (Network, IPC, PID) e os mantém ativos. Se um container da sua aplicação crashar e reiniciar, a rede e o endereço IP do Pod são preservados intactos graças ao `pause container`.

Ciclo de Vida de um Pod (*Pod Phase*):

- **`Pending`:** O manifesto foi aceito pelo API Server, mas o Pod ainda não teve seus containers criados (pode estar baixando imagens ou aguardando o Scheduler alocar um nó).
- **`Running`:** O Pod foi vinculado a um nó e todos os seus containers foram criados. Pelo menos um container está rodando ou em processo de inicialização/reinício.
- **`Succeeded`:** Todos os containers do Pod finalizaram suas execuções com sucesso (código de saída `0`) e não serão reiniciados (comum em *Jobs* de processamento batch).
- **`Failed`:** Todos os containers do Pod terminaram e pelo menos um encerrou com falha (código de saída diferente de `0`).
- **`Unknown`:** O estado do Pod não pôde ser obtido, geralmente por perda de comunicação entre o Control Plane e o `kubelet` do nó hospedeiro.

MÓDULO 4: CRIAÇÃO DO AMBIENTE PRÁTICO COM KIND (KUBERNETES IN DOCKER)

4.1 O QUE É O KIND?

O **Kind (Kubernetes in Docker)** é uma ferramenta mantida pela *Kubernetes Special Interest Group (SIGs)* projetada para rodar clusters Kubernetes locais utilizando **containers Docker como se fossem nós do cluster**.

Diferente de soluções pesadas que exigem a criação de múltiplas Máquinas Virtuais (como Minikube com VirtualBox), o Kind utiliza a própria engine do Docker local para simular os *Control Plane Nodes* e *Worker Nodes*. Isso resulta em uma inicialização extremamente rápida e baixo consumo de memória RAM.

```
Plaintext

MAQUINA HOST DO DESENVOLVEDOR (Linux / macOS / Windows)
  └── Docker Engine
        ├── Docker Container 1 (Simula o Nó: Control Plane do K8s)
        ├── Docker Container 2 (Simula o Nó: Worker 1 do K8s)
        └── Docker Container 3 (Simula o Nó: Worker 2 do K8s)
```

4.2 INSTALAÇÃO DO KIND E DEPENDÊNCIAS

Para executar os laboratórios a seguir, certifique-se de que o **Docker** e o **kubectl** estão instalados e ativos na sua máquina.

Comando de Instalação do Binary do Kind (Linux):

```
Bash

# Baixar o binário compilado do Kind
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.22.0/kind-linux-amd64

# Conceder permissão de execução
chmod +x ./kind

# Mover para o PATH do sistema
sudo mv ./kind /usr/local/bin/kind

# Validar a instalação
kind version
```

4.3 PROVISIONANDO UM CLUSTER MULTI-NODE COM KIND

Para simular um ambiente corporativo real de forma completa, vamos criar uma configuração do Kind contendo **1 Nó Control Plane e 2 Nós Worker**.

Passo 1: Criar o Arquivo de Configuração do Kind

Crie um arquivo chamado **`kind-cluster.yaml`**:

YAML

```yaml
# Arquivo de configuração do Kind para provisionar múltiplos nós
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

Passo 2: Executar a Criação do Cluster

Execute o comando do Kind passando o arquivo de configuração:

Bash

```bash
kind create cluster --name k8s-turma --config kind-cluster.yaml
```

Saída esperada no terminal:

```
Plaintext

Creating cluster "k8s-turma" ...
 ✓ Ensuring node image (kindest/node:v1.29.2) 🖼
 ✓ Preparing nodes 📦 📦 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane ———————— 🟢
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
 ✓ Joining worker nodes —— 🟢
Set kubectl context to "kind-k8s-turma"
You can now use your cluster with:

kubectl cluster-info --context kind-k8s-turma
```

MÓDULO 5: LABORATÓRIO PRÁTICO GUIADO (PASSO A PASSO)

Com o cluster Kind ativo e o contexto configurado automaticamente, iniciaremos a exploração prática dos comandos fundamentais da CLI `kubectl`.

5.1 ETAPA 1: INSPEÇÃO E MAPEAMENTO DO CLUSTER

Antes de implantar qualquer aplicação, um administrador Kubernetes precisa inspecionar a topologia e a saúde do cluster.

1. Verificar as versões do Cliente (`kubectl`) e do Servidor (Cluster):

```
Bash

kubectl version
```

> **O que observar:** *O comando exibe a versão do binário do `kubectl` instalado localmente (`Client Version`) e a versão exata do Kubernetes em execução no Control Plane (`Server Version`).*

2. Obter Informações de Conectividade do Control Plane e Serviços Básicos:

```bash
kubectl cluster-info
```

Saída Esperada:

```
Kubernetes control plane is running at https://127.0.0.1:41785
CoreDNS is running at https://127.0.0.1:41785/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

3. Listar os Nós Integrantes do Cluster e seus Status:

```bash
kubectl get nodes
```

Saída Esperada:

```
NAME                      STATUS   ROLES           AGE     VERSION
k8s-turma-control-plane   Ready    control-plane   2m40s   v1.29.2
k8s-turma-worker          Ready    <none>          2m18s   v1.29.2
k8s-turma-worker2         Ready    <none>          2m18s   v1.29.2
```

> **Análise Técnica:** *Note que temos 3 containers Docker operando como nós do Kubernetes. Todos estão no estado* `Ready` *(Prontos para aceitar workloads).*

4. Listar os Namespaces Existentes no Cluster:

```bash
kubectl get namespaces
```

(Você também pode utilizar o atalho oficial: `kubectl get ns`)

Saída Esperada:

```
NAME              STATUS   AGE
default           Active   3m10s
kube-node-lease   Active   3m10s
kube-public       Active   3m10s
kube-system       Active   3m10s
```

5. Listar os Pods Internos do Próprio Kubernetes:

Por padrão, `kubectl get pods` busca pods apenas no namespace `default` (que atualmente está vazio). Para listar os componentes do ecossistema do sistema:

```bash
kubectl get pods -n kube-system
```

> **O que observar:** *Você verá os Pods do* `coredns`, `kube-apiserver`, `kube-controller-manager`, `kube-proxy`, `kube-scheduler` *e do driver CNI de rede (*`kindnet`*).*

5.2 ETAPA 2: CRIAÇÃO DECLARATIVA DO PRIMEIRO POD

Iremos escrever um manifesto YAML declarativo para implantar um Pod executando um servidor Web Nginx de alta performance.

Passo 1: Criar o Manifesto `pod.yaml`

Crie um arquivo no seu diretório de trabalho com o nome **`pod.yaml`**:

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-web-servidor
  namespace: default
  labels:
    app: webserver
    ambiente: treinamento
spec:
  containers:
    - name: container-nginx
      image: nginx:1.25-alpine
      ports:
        - containerPort: 80
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
```

Passo 2: Aplicar o Manifesto Declarativo

Envie a especificação para o `kube-apiserver` usando o comando `kubectl apply`:

```bash
kubectl apply -f pod.yaml
```

Saída Esperada:

```
pod/pod-web-servidor created
```

5.3 ETAPA 3: INSPEÇÃO PROFUNDA E DIAGNÓSTICO (TROUBLESHOOTING)

Agora que o objeto Pod foi solicitado, aplicaremos os comandos fundamentais de diagnóstico do Kubernetes para verificar sua alocação,logs e execução.

1. Listar o Pod e Obter Informações Expandidas (`-o wide`):

```bash
kubectl get pods -o wide
```

Saída Esperada:

```
NAME              READY   STATUS    RESTARTS   AGE   IP           NODE                NOMINATED NODE   READINESS GATES
pod-web-servidor  1/1     Running   0          42s   10.244.1.2   k8s-turma-worker    <none>           <none>
```

Análise Técnica:

- `READY 1/1`: *Significa que o container declarante está rodando com sucesso.*
- `STATUS Running`: *O Pod passou pelas fases de* `Pending`*, baixou a imagem e está ativo.*
- `IP 10.244.1.2`: *Endereço IP Virtual interno atribuído exclusivamente ao Pod.*
- `NODE k8s-turma-worker`: *O* `kube-scheduler` *escolheu automaticamente o nó Worker 1 para executar este workload.*

2. Inspecionar os Detalhes Operacionais do Pod (`describe`):

O comando `describe` traz um raio-X completo do objeto, incluindo metadados, volumes, limites e, o mais importante, o histórico de **Eventos do Kubernetes**.

Bash

```
kubectl describe pod pod-web-servidor
```

Exemplo de Trecho Relevante da Saída (Eventos):

```
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  1m    default-scheduler  Successfully assigned default/pod-web-servidor to k8s-turma-worker
  Normal  Pulling    1m    kubelet            Pulling image "nginx:1.25-alpine"
  Normal  Pulled     58s   kubelet            Successfully pulled image "nginx:1.25-alpine" in 2.1s
  Normal  Created    58s   kubelet            Created container container-nginx
  Normal  Started    58s   kubelet            Started container container-nginx
```

💡 **Dica de Troubleshooting:** *Se um Pod ficar preso em* `CrashLoopBackOff` *ou* `ImagePullBackOff`*, o final da saída do* `kubectl describe pod` *indicará a causa exata do erro!*

3. Consultar os Logs da Aplicação (`logs`):

Para visualizar as saídas geradas na `stdout` e `stderr` do container dentro do Pod:

```bash
kubectl logs pod-web-servidor
```

Para acompanhar os logs em tempo real (*Follow Mode*):

```bash
kubectl logs -f pod-web-servidor
```

4. Executar Comandos Interativos dentro do Container (`exec`):

Assim como no Docker, podemos abrir uma sessão Shell dentro do Pod para inspecionar arquivos ou testar conexões locais.

```bash
kubectl exec -it pod-web-servidor -- sh
```

Dentro do container no terminal:

```bash
# Fazer uma requisição HTTP interna para a aplicação rodando em localhost
curl localhost:80

# Sair do container
exit
```

5.4 ETAPA 4: EXCLUSÃO E LIMPEZA DE RECURSOS

No Kubernetes, a exclusão de um recurso é declarativa e imediata no Control Plane.

Excluir o Pod passando o manifesto YAML:

```bash
kubectl delete -f pod.yaml
```

(Ou alternativamente, especificando o nome do Pod diretamente):

```bash
kubectl delete pod pod-web-servidor
```

Saída Esperada:

```
Plaintext

pod "pod-web-servidor" deleted
```

Valide que o Pod foi devidamente finalizado e removido:

```
Bash

kubectl get pods
```

Saída: `No resources found in default namespace.`

MÓDULO 6: GUIA DE CONSULTA RÁPIDA (CHEATSHEET)

Comandos `kubectl` Essenciais do Dia a Dia

```
# ================================================================================
# 1. INFORMAÇÕES DO CLUSTER E NÓS
# ================================================================================
kubectl cluster-info                    # Exibe os endpoints do Control Plane e DNS
kubectl get nodes                       # Listagem simples dos nós
kubectl get nodes -o wide                # Listagem detalhada dos nós (IPs, SO, Runtime)
kubectl describe node <nome-do-no>      # Detalhes de hardware, uso e taints do nó

# ================================================================================
# 2. GERENCIAMENTO DE NAMESPACES
# ================================================================================
kubectl get namespaces                  # Lista todos os namespaces
kubectl create namespace <nome>         # Cria um namespace de forma imperativa

# ================================================================================
# 3. GERENCIAMENTO E INSPEÇÃO DE PODS
# ================================================================================
kubectl apply -f <arquivo.yaml>         # Cria/Atualiza recursos declarativamente
kubectl get pods                        # Lista os pods no namespace atual
kubectl get pods -A                     # Lista todos os pods de TODOS os namespaces
kubectl get pods -o wide                # Exibe IPs dos Pods e em quais Nós estão rodando
kubectl describe pod <nome-do-pod>      # Exibe detalhes e histórico de eventos do Pod
kubectl logs <nome-do-pod>              # Exibe as linhas de log do Pod
kubectl logs -f <nome-do-pod>           # Acompanha o log do Pod em tempo real
kubectl exec -it <nome-do-pod> -- sh    # Abre um Shell interativo dentro do Pod

# ================================================================================
# 4. REMOÇÃO DE RECURSOS
# ================================================================================
kubectl delete -f <arquivo.yaml>        # Remove os objetos definidos no manifesto YAML
kubectl delete pod <nome-do-pod>        # Remove um Pod específico do cluster
```

Glossário dos Principais Status de Erro em Pods:

- **`ContainerCreating`:** O `kubelet` está configurando os volumes, a rede e baixando as imagens do container.
- **`CrashLoopBackOff`:** O container iniciou, mas seu processo principal encerrou com falha repetidamente. O Kubernetes tenta reiniciar o container aplicando um tempo de espera progressivo (*backoff*).

- **`ImagePullBackOff` / `ErrImagePull`:** O Kubernetes não conseguiu baixar a imagem especificada no manifesto (nome de imagem errado, tag inexistente ou falta de credenciais do registro de imagens).
- **`Pending`:** O Pod não pôde ser agendado em nenhum nó (falta de recursos de CPU/Memória nos nós, ou restrições de afinidade não atendidas).

REFERÊNCIAS

Livros e Obras de Referência

BURNS, Brendan; BEDA, Joe; HIGHTOWER, Kelsey; EVENSON, Lachlan. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. 3. ed. Sebastopol: O'Reilly Media, 2022.

IBRYAM, Bilgin; HUß, Roland. *Kubernetes Patterns: Reusable Elements for Designing Cloud-Native Applications*. 2. ed. Sebastopol: O'Reilly Media, 2023.

HIGHTOWER, Kelsey; BURNS, Brendan; BEDA, Joe. *Kubernetes: Up and Running*. 2. ed. Sebastopol: O'Reilly Media, 2019.

LUKSA, Marko. *Kubernetes in Action*. 2. ed. Manning Publications, 2023.

MAYOR, Michael; HERNANDEZ, Loris. *Kubernetes Security and Observability*. Packt Publishing, 2022.

PULTORAK, David. *Docker and Kubernetes for Java Developers*. Packt Publishing, 2021.

SAYFAN, Gigi. *Mastering Kubernetes*. 4. ed. Packt Publishing, 2023.

Recursos Web e Documentação Oficial

CNCF — CLOUD NATIVE COMPUTING FOUNDATION. *Cloud Native Computing Foundation*. Disponível em: https://www.cncf.io/ . Acesso em: 11 out. 2026.

HELM. *Helm Documentation: The Package Manager for Kubernetes*. Disponível em: https://helm.sh/docs/ . Acesso em: 11 out. 2026.

KUBERNETES. *Kubernetes Documentation*. Disponível em: https://kubernetes.io/docs/ . Acesso em: 11 out. 2026.

KUBERNETES. *Concepts: Cluster Architecture, Workloads, Networking, Storage and Security*. Disponível em: https://kubernetes.io/docs/concepts/ . Acesso em: 11 out. 2026.

KUBERNETES. *Tasks: Tutorials and Administrative Procedures*. Disponível em: https://kubernetes.io/docs/tasks/ . Acesso em: 11 out. 2026.

KUBERNETES. *kubectl Reference Documentation*. Disponível em: https://kubernetes.io/docs/reference/kubectl/ . Acesso em: 11 out. 2026.

MINIKUBE. *Minikube Documentation*. Disponível em: https://minikube.sigs.k8s.io/docs/ . Acesso em: 11 out. 2026.

PROMETHEUS. *Prometheus Documentation*. Disponível em: https://prometheus.io/docs/ . Acesso em: 11 out. 2026.

Referências Complementares para DevOps e Cloud Native

ARGO PROJECT. *Argo CD Documentation*. Disponível em: https://argo-cd.readthedocs.io/ . Acesso em: 11 out. 2026.

CNCF. *Cloud Native Landscape*. Disponível em: https://landscape.cncf.io/ . Acesso em: 11 out. 2026.

DOCKER. *Docker Documentation*. Disponível em: https://docs.docker.com/ . Acesso em: 11 out. 2026.

OPEN CONTAINER INITIATIVE (OCI). *Image Specification and Runtime Specification*. Disponível em: https://opencontainers.org/ . Acesso em: 11 out. 2026.

SIGSTORE. *Sigstore Documentation*. Disponível em: https://docs.sigstore.dev/ . Acesso em: 11 out. 2026.